

CUSTOMISED ACCELERATORS For Domain-Specific Computing Using FPGAs

There is always room for improvement in the world of technology. But it can only be improved and never perfected. System processors have seemingly come a long way, arguably even peaked. Let us see if that is really the case or not



JASON CONG,
Volgenau Chair
for Engineering
Excellence, and
Director of VAST,
UCLA

If you look at the performance and the growth of general-purpose computers, you will see that they had a spectacular growth from the 1980s all the way to 2000. But thereafter it slowed down considerably to 12% a year, and currently the rate is down to 3% a year.

But there still seems to be a lot of room available where you can make computer architecture adapt to do domain-specific customisations. We submitted a proposal to the National Science Foundation US, back in 2008, which got us the funding to establish a centre for domain-specific computing.

The goal at the time was to look beyond parallelisation and focus on domain-specific customisation to bring significant power-performance efficiency. Recently we have released a paper which talks about the research that has been done in the past decade.

Successful examples of customisation

The best possible example of customisation is the Google Tensor Processing Unit (TPU). Its first version was released in 2017. It was subsequently upgraded from DDR3 to DDR5 where the bandwidth went up to 188 gigabytes per second. At the time it was 200 times faster than a general-purpose central processing unit (CPU) and 70 times more energy-efficient.

In the last five years both the CPUs and graphics processing units (GPUs) have improved significantly. The problem however is that not everyone can do such a design, because doing such designs requires mil-

lions of dollars. It can take 12 to 24 months on average to build a project like this.

Customisation for the masses using FPGAs

Everyone wants to do customisation today for which we can use field programmable gate arrays (FPGAs). They consist of a lot of configurable logic blocks or lookup tables which can be customised to implement logic functions. They also have a number of dedicated components, for example, memory/multipliers and I/O processors, which provide additional efficiency.

If you have generated bit streams then you can reprogram FPGAs in seconds and get new functionality. They are also very affordable. The only shortcoming that the FPGAs have is that they have lower density and clock rate compared to application specific integrated circuits (ASICs). Let us now take a look at a few examples.

Example 1. Scalable sorting. We designed a multiple FPGA based sorting entrance. Here is one that we have for a D1 level sorting:

Phase 1: Multiple merge trees work with multiple HBM channels concurrently.

Phase 2: Merge trees are reused from phase 1 with extra logic to build a wider tree.

As a result, we can sort four gigabytes of data in 0.25 seconds.

Example 2. This is a case from Microsoft. In this case they have an FPGA that is considered as a “bump-in-the-wire” before they connect to that data centre switch. This allows them to do a lot of on-the-fly compu-